

params_schema.py — Full Parameter Reference

This document explains every parameter in `params_schema.py`, organized by section, with dedicated deep-dives on Monte Carlo configuration and the scenario system that lets different inflow/stock/outflow assumptions be modeled.

1. Why this file exists, and how it's structured

`params_schema.py` defines every parameter as a **frozen dataclass** (immutable — you can't accidentally mutate a `Params` object after creation; changing a value means building a new one via `dataclasses.replace()`). It lives in `src/`, not in `00_parameters.py` itself, specifically so pickling/unpickling the `params` artifact works correctly across different stage scripts (each run as their own `__main__`) — see the file's own top docstring for the full explanation.

The top-level object is `Params`, with **seven sections**, one per stage (plus one cross-cutting):

```
@dataclass(frozen=True)
class Params:
    data_prep: DataPrepParams           # stage 01
    stock_flow: StockFlowParams         # stage 02
    adjusted_flows: AdjustedFlowsParams # stage 03_02
    disaggregation: DisaggregationParams # stage 03 (03_01)
    materials: MaterialsParams          # stage 04
    visualization: VisualizationParams  # stage 06
    monte_carlo: MonteCarloParams       # cross-cutting, used by
02/03_01/03_02
```

Every section (and several nested objects within them) has a `.validate() -> list[str]` method — `Params.validate()` calls all of them and also checks cross-section consistency (e.g. that `data_prep.end_year_model` and `stock_flow.model_end_year` agree). An empty list means no issues found; `00_parameters.py` prints any issues to `stderr` but does **not** stop the run — validation is advisory, not a hard gate.

`Params.to_nested_dict()` gives a legacy dict view (`{"01_data_prep": {...}, ...}`) for the small number of call sites that still expect the old `PARAMS["section"]["key"]` style rather than `params.section.key` attribute access.

2. DataPrepParams (stage 01 — 01_data_prep.py)

Controls REMIND scenario loading and EU export-data cleaning.

Field	Meaning
<code>scenario</code>	Which REMIND scenario is actually used downstream (must be a key in <code>remind_scenario_files</code> and a member of <code>scenario_list</code>). Default <code>"npi25"</code> .
<code>scenario_list</code>	The full set of scenario names this project recognizes.
<code>remind_scenario_files</code>	<code>{scenario_name: (relative_path_to_.mif, csv_separator)}</code> — where to find each scenario's raw REMIND output and what delimiter it uses.
<code>start_year_model / end_year_model</code>	The full model horizon (1900–2070 by default). Cross-checked against <code>stock_flow.model_end_year</code> and <code>disaggregation.end_year_model</code> — <code>Params.validate()</code> flags a mismatch.
<code>start_year_plotting / end_year_plotting</code>	Separate, narrower window used only for default chart x-axes (2015–2070) — doesn't affect any computation.
<code>composition_extend_from_year</code>	Year from which stage-04 composition data extrapolation begins.

<code>accelerating_year</code>	Used in the BEV-transition-warping diagnostic (<code>warp_bev_transition_all_segments</code>). Flagged as unresolved (finding L2) – no cited source for why 2026 specifically.
<code>threshold</code>	Numerical tolerance for stock-flow stability/consistency checks.
<code>prefix</code>	The REMIND <code>Variable</code> taxonomy prefix stage 01 filters on (<code>"Stock\ Transport\ Pass\ Road\ LDV"</code>).
<code>eu_countries</code>	The 27 EU country names used for regional aggregation of export data.
<code>remind_regions</code>	The 10 REMIND region codes making up <code>"EUR"</code> in this analysis.
<code>remind_technology</code>	REMIND's own coarse drivetrain categories (BEV, Hybrid, Liquids, Gases, FCEV) – not the finer split used later.
<code>target_technology</code>	The finer drivetrain split this project actually models (BEV, HEV, PHEV, Petrol, Diesel) – the split from Hybrid→HEV / PHEV and Liquids→Petrol / Diesel happens in stage 03, from EEA data.
<code>target_class_detail</code>	REMIND's vehicle-class-detail labels this project keeps.
<code>attribute_list</code> , <code>key_names</code>	Column-naming conventions used when reshaping REMIND's wide format.
<code>element_list</code> , <code>element_list_noAlCu</code>	The 9 tracked elements (Ag, In, Ta, Zn, Dy, Nd, Pr, Al, Cu) and a variant excluding Al/Cu – used by stage 04's element-level path.

<code>input_dir / output_dir</code>	Relative paths (resolved against the <i>script's own location</i> , not the caller's cwd — see <code>01_data_prep.py</code> 's own fix note).
<code>composition_file_name,</code> <code>petrol_composition_file_name</code>	Raw composition workbook filenames (stage 04, main + Petrol-specific).
<code>export_data_file_name</code>	"usedvehicles_v1.2.xlsx" — the raw EU used-vehicle trade data.
<code>sheets</code>	Sheet names expected inside the composition workbook.
<code>stock_interpolation_method</code>	"cubic" or "pchip" — how REMIND's discrete-year stock gets interpolated to annual resolution.
<code>export_min_year /</code> <code>export_max_year_exclusive</code>	Year window kept from the raw export/import trade data ([2005, 2023)).
<code>iso3_corrections</code>	Manual fixes for known-bad ISO3 codes in the raw trade data (currently just "IRE" → "IRL").

3. StockFlowParams (stage 02 — `02_stockdriven.py`, and the base every later stage's point estimates build on)

This is the section with the **base point estimates** for vehicle lifetime and destination shares — and the Monte Carlo *spread* parameters built directly around them (see §5 for the full Monte Carlo picture).

3.1 Point estimates

Field	Meaning
-------	---------

<code>model_end_year</code>	2070 — must match <code>data_prep.end_year_model</code> (validated).
<code>last_exp_data_year</code>	Last year with observed export data (2022).
<code>init_max_age</code>	50 — how many years back the cohort backcast extends before the model horizon starts. Centralized here (was a hardcoded local in <code>02_stockdriven.py</code>) specifically because stage 03_01's Monte Carlo needs the <i>identical</i> value to regenerate stage 02's per-draw results from saved <code>scale_lambda</code> draws — if these drifted apart between two files, stage 03_01 would silently regenerate the wrong cohort ages.
<code>lifetime_by_drv</code>	<code>dict[str, WeibullLifetime]</code> — the actual survival curve per drivetrain: <code>WeibullLifetime(shape_k, scale_lambda)</code> . BEV and the liquid/gas/hydrogen drivetrains default to <code>scale_lambda=13.0</code> ; HEV/PHEV/Hybrid default to <code>9.0</code> . <code>shape_k=3.0</code> everywhere (shape is never made uncertain anywhere in this codebase — only <code>scale_lambda</code> , the characteristic lifetime, varies).
<code>lifetime_override_by_drv</code>	<code>dict[str, LifetimeOverride None]</code> — an optional <i>windowed</i> override (<code>start_year</code> , <code>end_year</code> , <code>shape_k</code> , <code>scale_lambda</code>) per drivetrain, used by stage 02's <code>get_effective_lifetime_params</code> . All <code>None</code> by default. This is a different, coexisting convention from <code>AdjustedFlowsParams</code> 's <code>OpenEndedLifetimeChange</code> (§4) — this one is bounded (has an <code>end_year</code>), that one is open-ended (no <code>end_year</code> , matches what <code>flowdriven_model.py</code> actually implements). Don't confuse the two.
<code>unknown_whereabouts_share</code>	Point estimate: share of retired vehicles whose fate is untracked, per drivetrain. BEV: <code>0.10</code> ; everything else: <code>0.43</code> .
<code>export_share_by_drv</code>	Point estimate: share of retired vehicles assumed exported, per drivetrain. BEV: <code>0.02</code> ; everything else: <code>0.08</code> (FCEV/Gases explicitly flagged as unverified placeholders).

negative_inflow_policy	"report_only" (default) or "clip_to_target" — how 02_stockdriven.py handles a year where the stock-driven model's residual-inflow formula goes negative (prescribed stock falling faster than natural attrition explains). "report_only" lets modeled stock exceed the falling target from then on; "clip_to_target" forces extra pro-rata outflow to hit the target exactly. Both implemented; "report_only" remains the default since switching changes real output numbers.
------------------------	--

3.2 The uncertainty spreads (feed Monte Carlo — see §5.2 for the full mechanism)

Field	Meaning
lifetime_scale_lambda_relative_spread	dict[str, float \ AsymmetricSpread] — how uncertain each drivetrain's scale_lambda is. A plain float means symmetric ($\pm$that fraction); an AsymmetricSpread(lower=..., upper=...) allows genuinely different lower/upper bounds. Placeholder 0.15 ($\pm 15\%$) for every drivetrain today — not derived from any real uncertainty estimate, tune once you have one (e.g. from a survival-curve fit's own confidence interval).
unknown_whereabouts_share_std	dict[str, float] — standard deviation for the Normal distribution around unknown_whereabouts_share. Placeholder 0.05 for every drivetrain.
export_share_std	dict[str, float] — same, around export_share_by_drv. Placeholder 0.02.

Why spreads, not baked-in distributions? These are stored as relative spreads/standard deviations, not `Distribution` objects — the actual distribution is built fresh from whatever the *current* point estimate is at sampling time. Change `lifetime_by_drv["BEV"].scale_lambda` and the Monte Carlo spread around it updates automatically; there's no separate "distribution center" value that could drift out of sync with the point estimate.

4. AdjustedFlowsParams (stage 03_02 — 03_02_adjustedflows.py) — the scenario / inflow-stock-outflow system

This is the section that answers "how do I model a different inflow, stock, or outflow assumption?" — full walkthrough below.

4.1 The core idea: ScenarioSpec

Every one of the 11 built-in scenarios (BAU, BEV_only, BEV_A_F, BEV_JA_JF, BEV_large, BEV_small, BEV_longer, ICEV_shorter, stock_lower, losses_zero, losses_high) is **one** `ScenarioSpec` **object**, living in `AdjustedFlowsParams.scenarios: dict[str, ScenarioSpec]`. Nothing about a scenario is hardcoded anywhere in `03_02_adjustedflows.py` itself — the whole script just loops over this dict.

A `ScenarioSpec` has five kinds of override, and every one of them is **SPARSE and ADDITIVE** over `StockFlowParams`'s base point estimates: leaving a field at its default (`None` or empty dict) means "use the base value," not "zero it out." This is what lets `ICEV_shorter` specify *only* Diesel and Petrol's changed lifetime without having to also restate BEV's unchanged one.

```
@dataclass(frozen=True)
class ScenarioSpec:
    name: str

    # --- changes WHAT COMES IN (inflow composition) ---
    inflow_drivetrain_shares_final: dict[str, float] | None = None
    inflow_segment_shares_drivetrain: str | None = None
    inflow_segment_shares_final: dict[str, float] | None = None
```

```

# --- changes HOW LONG VEHICLES LAST (affects both stock and outflow
timing) ---
lifetime_change_by_drv: dict[str, OpenEndedLifetimeChange] =
field(default_factory=dict)

# --- changes WHERE OUTFLOW GOES (destination split, not timing) ---
export_share_overrides: dict[str, float] = field(default_factory=dict)
unknown_whereabouts_share_overrides: dict[str, float] =
field(default_factory=dict)

# --- changes HOW MANY COME IN, as a flat multiplier ---
stock_modifier: float = 1.0

```

4.2 Changing inflow — two mutually-exclusive transforms

- `inflow_drivetrain_shares_final`: changes the overall **drivetrain mix** of inflow (e.g. `BEV_only` sets `{"BEV": 1.0, "HEV": 0.0, "PHEV": 0.0, "Diesel": 0.0, "Petrol": 0.0}`). Maps onto `tweak_inflow_drivetrain_shares()`.
- `inflow_segment_shares_drivetrain` + `inflow_segment_shares_final`: changes **one drivetrain's internal segment mix** (A–F, JA–JF), holding the overall drivetrain mix fixed. Used by `BAU` (its own segment-mix baseline — yes, `BAU` applies its own explicit segment reshaping, not a no-op) and the four BEV segment-profile scenarios (`BEV_A_F`, `BEV_JA_JF`, `BEV_large`, `BEV_small`). Maps onto `tweak_inflow_segment_shares_within_drivetrain()`.

At most one of these should be set per scenario (validated). Neither set means "reuse `BAU`'s already-resolved inflow unchanged" — this is why `BAU` must always exist in `scenarios` (validated) and is processed first: every scenario without its own inflow transform falls back to `BAU`'s result.

Both transforms ramp from the current share at `scenario_start_year` to the target share by `scenario_ramp_end_year` (both 2026 today — an immediate switch, no actual ramp, but kept as two separate fields since nothing requires them to be equal if a future scenario wants a genuine multi-year transition).

4.3 Changing how long vehicles last — `lifetime_change_by_drv`

A sparse `dict[str, OpenEndedLifetimeChange]`. `OpenEndedLifetimeChange(start_year, shape_k, scale_lambda)` has **no** `end_year` — it applies from `start_year` onward for the rest of

the simulation, which is genuinely what `flowdriven_model.py` implements (confirmed from source: `if t >= lifetime_change_by_drv[drv]["start_year"]`, no `end_year` ever read). This is a different type from `StockFlowParams`'s `LifetimeOverride` (§3.1), which is *windowed* (has both start and end) — two legitimately different, coexisting conventions in this codebase.

Every entry's `start_year` should match `AdjustedFlowsParams.lifetime_change_start_year` (2027) — validated, since all lifetime-change scenarios are expected to share one synchronized switch year.

This is the mechanism behind `BEV_longer` (BEV: `scale_lambda=17`, vs. the base 13) and `ICEV_shorter` (Diesel/Petrol: `scale_lambda=9`).

Because this affects the actual survival curve, it changes both stock AND outflow simultaneously — longer-lived vehicles retire less (lower outflow) and therefore accumulate more stock, within the same simulated horizon. This is distinct from the two "destination" overrides below, which only redirect where outflow *goes*, not how much of it there is.

4.4 Changing where outflow goes — `export_share_overrides` / `unknown_whereabouts_share_overrides`

Sparse `dict[str, float]` overrides on top of `StockFlowParams.export_share_by_drv` / `.unknown_whereabouts_share`, for one or more drivetrains. Used by `losses_zero` (BEV's both forced to `0.0` — everything retiring gets collected) and `losses_high` (BEV's raised to match every other drivetrain's `0.08 / 0.43`). These do **not** change how much outflow there is overall — only how a fixed amount of outflow splits into collected/exported/unknown-whereabouts.

4.5 Changing how many vehicles come in — `stock_modifier`

A flat multiplier applied to every drivetrain/segment's inflow from `AdjustedFlowsParams.stock_modifier_start_year` (2027) onward. `stock_lower` sets this to `0.8` (20% less new-vehicle inflow from 2027). Deliberately independent from `lifetime_change_start_year` — "how many vehicles enter the fleet" and "how long vehicles last" are conceptually unrelated scenario dimensions that just happen to default to the same year today.

4.6 Shared, non-scenario-specific fields

Field	Meaning
<code>scenario_start_year / scenario_ramp_end_year</code>	2026 / 2026 — shared timing window for every inflow-composition transform (§4.2).
<code>lifetime_change_start_year</code>	2027 — shared switch year every <code>lifetime_change_by_drv</code> entry must match (validated).
<code>stock_modifier_start_year</code>	2027 — shared switch year for every scenario's <code>stock_modifier</code> .
<code>base_year</code>	2005 — the stock-flow cohort base year: each scenario rebuilds its starting cohort stock from stage 02's matrix AT this year (not a fresh 1975 backcast). Must precede <code>scenario_start_year</code> (validated).

4.7 Selecting which scenarios to run — `scenarios_to_run`

By default, `03_02_adjustedflows.py` simulates **all 11 scenarios** in `scenarios` — each one a full deterministic run plus (when `monte_carlo.enabled`) its own Monte Carlo pass. For focused research (testing one idea, iterating quickly) that's often more than you need: `scenarios_to_run` lets you narrow a run down to just the scenario(s) you actually care about, without deleting or commenting out entries in `scenarios` itself.

```
scenarios_to_run: tuple[str, ...] | None = ("BAU",)
```

Field	Meaning
<code>scenarios_to_run</code>	<code>tuple[str, ...] \ None</code> . <code>None</code> means "run every scenario in <code>scenarios</code> " (the full 11-scenario sweep). A tuple of names restricts the run to just those. Default is <code>("BAU",)</code> — a fast, minimal run out of the box; widen it deliberately when you need more.

Valid names (must be a key in `scenarios`, checked in `validate()`):

BAU, BEV_only, BEV_A_F, BEV_JA_JF, BEV_large, BEV_small, BEV_longer, ICEV_shorter, stock_lower, losses_zero, losses_high.

An unknown name or an empty tuple is caught by `AdjustedFlowsParams.validate()` at parameter-build time (`00_parameters.py`) — a typo fails fast, not partway through a multi-hour run.

Resolving the field: don't re-implement the `None`-vs-tuple logic at the call site —

`AdjustedFlowsParams.active_scenario_names()` does this once, returning the actual list to simulate (in `scenarios`' own BAU-first order, which the inflow-resolution step in `03_02_adjustedflows.py` relies on):

```
active = p03_02.active_scenario_names()
# scenarios_to_run=None          -> all 11, in `scenarios` order
# scenarios_to_run=("BAU",)      -> ("BAU",) [default]
# scenarios_to_run=("stock_lower", "BAU") -> ("BAU", "stock_lower")
[reordered]
```

Important nuance: regardless of this selection, BAU's own resolved **inflow** is always computed by `03_02_adjustedflows.py` (every scenario without its own inflow transform reuses it) — but BAU is only actually **simulated** (deterministic + Monte Carlo run, tracker/output produced) if "BAU" is itself included in `scenarios_to_run`. Leaving BAU out of the selection is safe and saves time if you don't need its own output that run.

How to change this for your own run: this field lives on a frozen dataclass with a shared default, used by everyone running the pipeline — don't edit the literal default above for a one-off run, since that changes what "just run `00_parameters.py` with no overrides" means for every other scientist using this codebase, and will cause merge conflicts. Instead, override it wherever `Params` / `AdjustedFlowsParams` is actually constructed for your run (typically `00_parameters.py`):

```
params = Params(
    adjusted_flows=AdjustedFlowsParams(
        scenarios_to_run=("BAU", "stock_lower"),
    )
)
```

)

Only change the literal default in `params_schema.py` if you're deliberately changing what the no-overrides case means for everyone.

Note: `monte_carlo.enabled` is a separate, cross-cutting switch shared by stages 02, 03_01, and 03_02 (§5.1) — narrowing a 03_02 run with `scenarios_to_run` does not touch it, so stages 02 and 03_01 keep running Monte Carlo as usual either way.

5. Monte Carlo — the full picture

Monte Carlo configuration is split across **two places**, and it's easy to conflate them — they are NOT the same thing:

- `MonteCarloParams` (cross-cutting): the on/off switch, draw count, seed, memory-chunking, and the shared `output_periods` list. Does **not** itself define how uncertain any given quantity is.
- `StockFlowParams`'s three spread fields (§3.2): the actual uncertainty *magnitudes* that stages 02, 03_01, and 03_02 all sample from.

5.1 `MonteCarloParams` fields

Field	Meaning
<code>enabled</code>	Master on/off switch. <code>True</code> by default. When <code>False</code> , every stage's deterministic output is completely unaffected — Monte Carlo is purely additive, never changes a deterministic result.
<code>n_draws</code>	2000 — draw count used by stages 02, 03_01, and 03_02's <i>real</i> Monte Carlo passes (via <code>StockFlowParams</code> 's spreads).

seed

42 — top-level seed. Each stage/scenario/drivetrain spawns its own independently-seeded child generator from this one seed (via `numpy.random.SeedSequence.spawn()`), so nothing's draws are accidentally correlated with another's by sharing a raw stream.

`stockflow_n_draws / stockflow_seed /`
`stockflow_lifetime_spread /`
`stockflow_share_spread`

A separate, lighter-weight parameter set for a standalone full-uncertainty demo script (`mc_stockflow_uncertainty.py`) that varies lifetime + both shares for every stage-02 drivetrain simultaneously, at a much larger default draw count (200,000). Kept deliberately separate from `n_draws` above so enabling the everyday stage 02/03_01/03_02 Monte Carlo never accidentally triggers a 200,000-draw run meant for that other script. `stockflow_lifetime_spread /` `stockflow_share_spread` are single flat floats (not per-drivetrain dicts) — don't confuse these with `StockFlowParams.lifetime_scale_lambda` `_relative_spread` etc., which are the per-drivetrain dicts stages 02/03_01/03_02 actually read.

chunk_size

20,000 — draws are processed in batches of this size in the vectorized engine (`cohort_flow_mc.py`), so peak memory is bounded by `chunk_size × n_cohorts`, not `n_draws × n_cohorts`. A performance knob, not a model assumption.

output_periods

list[tuple[int, int]], default [(1975, 2070)] — which (start_year, end_year) INCLUSIVE windows to report results for. A single year is just a range where start == end (e.g. (2030, 2030)). Shared across stages 02, 03_01, and 03_02 — specifying it once here keeps all three consistent by construction. Specified up front, not queryable after a run completes (this is what keeps memory bounded at 200,000 draws — see §5.3).

5.2 What actually gets sampled, and from which distribution

For each drivetrain (stages 02/03_01) or each drivetrain within each scenario (stage 03_02), three quantities are resampled per Monte Carlo draw:

```
scale_lambda_draw ~ Triangular(point*(1 - lower), point, point*(1 + upper))
# lower = upper = the same float when
lifetime_scale_lambda_relative_spread[drv]
# is a plain float (the default everywhere today); genuinely different
lower/upper
# when it's an AsymmetricSpread.

unknown_whereabouts_share_draw ~ Normal(point, std), clipped to [0, 1]
export_share_draw                ~ Normal(point, std), clipped to [0, 1]
```

Why Triangular for lifetime but Normal for shares? This is a real, deliberate distinction, not an inconsistency — Triangular naturally supports asymmetric bounds (useful for lifetime, where "longer is more plausible than shorter" is a real belief worth representing), while the two shares use a standard clipped Normal. Mixing them up (e.g. using Normal for lifetime) was an actual bug caught and fixed earlier.

A subtlety worth knowing: with an *asymmetric* Triangular, the sampled distribution's mean is **not**

exactly the point estimate — it's $(\text{low} + \text{mode} + \text{high}) / 3$, which shifts toward whichever side has the wider spread. This is correct distribution behavior (if you believe "more likely to run longer than shorter," the mean *should* sit above the point estimate), not a bug — but it can look surprising in output summaries if you're not expecting it. Symmetric spreads are unaffected (mean stays exactly at the point estimate).

Shape (shape_k) is never made uncertain anywhere — only the characteristic lifetime (scale_lambda) varies.

5.3 Where the uncertainty actually gets applied, per stage

- **Stage 02:** only scale_lambda varies (one uncertain input per drivetrain — unknown_whereabouts_share / export_share don't apply yet, since stage 02 doesn't split outflow into destinations). Stock is fully **deterministic** here — it's pinned exactly to the REMIND-prescribed target every year regardless of the lifetime draw, so "cumulative stock" for a period is reported as a distribution-shaped summary with std=0 by construction, not a genuine spread.
- **Stage 03_01:** reuses stage 02's saved scale_lambda draws (loaded from the mc_stage02_draws artifact — the same draws, not independently resampled, which is what makes this genuine propagation rather than an independent re-guess), and additionally samples its own unknown_whereabouts_share / export_share draws. Stock remains deterministic (same stock-driven paradigm as stage 02).
- **Stage 03_02:** genuinely re-simulates each scenario's own scale_lambda, unknown_whereabouts_share, and export_share per draw (via the vectorized cohort_flow_mc.py engine) — including carrying a scenario's lifetime_change_by_drv override through correctly (the same per-draw multiplier that produced "10% longer" under the base regime stays "10% longer" after a scenario's override kicks in, rather than resampling independently at the switch year). **Stock is genuinely uncertain here** (unlike stages 02/03_01), since 03_02's flow-driven model doesn't pin stock to an external target — it's a real output of the recurrence, which does vary with the sampled lifetime.

5.4 output_periods and the histogram/summary output

Every requested period, for every applicable scope (EU-total, per-drivetrain, and — stage 03_02 only — per-segment), gets a full distribution summary: n, mean, median, mode, std, p2_5, p97_5, min, max, plus a 50-bin histogram (bin_edges, frequencies). This is what monte_carlo.summarize_distribution() computes, and what the Excel export

(`export_mc_summaries_excel.py`) turns into spreadsheet form — one flat "Summary" sheet plus one sheet per histogram.

For a multi-year period, three additional stock-specific views are computed: `stock_end_of_period` (snapshot at the period's last year), `stock_sum_over_period` ("stock-years" — summed across every year in the window), and `stock_per_year` (each individual year in the window reported separately). A single-year period collapses all three to the same value.

This is specified up front, not queryable after a run completes — stage 03_02's vectorized engine accumulates exactly these windows *during* its one simulation pass (critical for staying memory-bounded at 200,000 draws — a full per-cohort history across all years would be prohibitively large). Stages 02/03_01 already track cheap full per-year (not per-cohort) arrays and sum over the requested windows afterward.

6. `DisaggregationParams` (stage 03 — `03_01_flowdriven.py`)

Field	Meaning
<code>output_dir</code>	Where stage 03 writes its outputs.
<code>start_year_model / end_year_model</code>	Must match <code>data_prep</code> 's horizon (validated).
<code>use_synthetic_eea_fallback</code>	<code>False</code> by default — your real <code>EEA_final_data.csv</code> governs the HEV/PHEV and Diesel/Petrol splits. Set <code>True</code> only as a temporary bridge while waiting for that file; <code>03_01_flowdriven.py</code> will then generate a clearly-labeled synthetic placeholder instead of raising. Should almost always stay <code>False</code> once the real file exists.
<code>synthetic_eea_seed</code>	Seed for that synthetic fallback generator.

7. MaterialsParams (stage 04)

Field	Meaning
<code>segment_map</code>	The 12 vehicle segment codes (A-F, JA-JF) mapped to their composition-dataset codes.
<code>drv_prefix_map</code>	Drivetrain → composition-dataset key prefix.
<code>region</code>	"EUR".
<code>drivetrains</code>	The 5 drivetrains stage 04 materializes composition for.
<code>composition_parameter_code</code>	"m-c" (material/component level, the required default) or "e-m" (element level). Controls how deep <code>quantify_elements_from_tracker()</code> decomposes composition — per explicit requirement, composition must stop at component+material, not decompose into individual chemical elements.
<code>material_level_key</code>	Which material-hierarchy depth to report at when using "m-c" — "materialKeyLevel_highest" by default (a per-row fallback to the deepest available level, matching the real stage-04 scripts' own usage).
<code>traction_composition_file_name</code> , <code>battery_share_file_name</code> , <code>battery_composition_file_name</code>	Centralized filenames for the traction-motor and battery workbooks (previously hardcoded inside their respective stage-04 scripts).

battery_composition_parameter_code	"e-m" — flagged as an unresolved conflict with the "material, not element" requirement above; the battery workbook's equivalent material-level code hasn't been confirmed yet (validation surfaces this as a standing reminder, not a hard error).
battery_size_map	Per-segment battery capacity (kWh) assumption — no year-dependence modeled (flagged as a known simplification).
average_battery_capacity_kwh	60.0 — fallback/reference value.

8. VisualizationParams (stage 06)

Field	Meaning
year_plot_start / year_plot_end	Default chart x-axis window (2015–2070).
year_ratio_start	Start year for ratio-based comparison charts.
element_list / element_list_noAlCu	Same element lists as DataPrepParams, reused here for chart legends/filters.
scenario	Which REMIND scenario stage 06's charts default to showing.

Quick reference: "I want to change X, where do I look?"

You want to...	Look at
----------------	---------

Change a drivetrain's base lifetime or destination shares	<code>StockFlowParams.lifetime_by_drv /</code> <code>.unknown_whereabouts_share /</code> <code>.export_share_by_drv</code>
Add a new sensitivity scenario	<code>AdjustedFlowsParams.scenarios</code> — add one new <code>ScenarioSpec</code> entry
Run only some scenarios, not all 11 (faster, focused research)	<code>AdjustedFlowsParams.scenarios_to_run</code>
Change how uncertain lifetime/shares are	<code>StockFlowParams.lifetime_scale_lambda</code> <code>_relative_spread /</code> <code>.unknown_whereabouts_share_std /</code> <code>.export_share_std</code>
Turn Monte Carlo on/off, or change draw count	<code>MonteCarloParams.enabled /</code> <code>.n_draws</code>
Get results for a specific year or year-range	<code>MonteCarloParams.output_periods</code>
Make lifetime uncertainty asymmetric for one drivetrain	<code>AsymmetricSpread(lower=...,</code> <code>upper=...) inside</code> <code>StockFlowParams.lifetime_scale_lambda</code> <code>_relative_spread</code>
Change which EEA data source is used	<code>DisaggregationParams.use_synthetic_eea_fallback</code>
Change composition decomposition depth	<code>MaterialsParams.composition_parameter</code> <code>_code /</code> <code>.material_level_key</code>